



Supervised Learning: Regression

A comprehensive guide to regression techniques, model evaluation, regularization, and hyperparameter tuning — designed for clarity and practical understanding.

Dr. Anilkumar V. Brahmane | Computer Engineering | Sanjivani College of Engineering

Agenda



Types of Regression

Linear (Simple & Multiple), Non-Linear, Polynomial Regression



Regularization Techniques

Lasso, Ridge, ElasticNet Regression



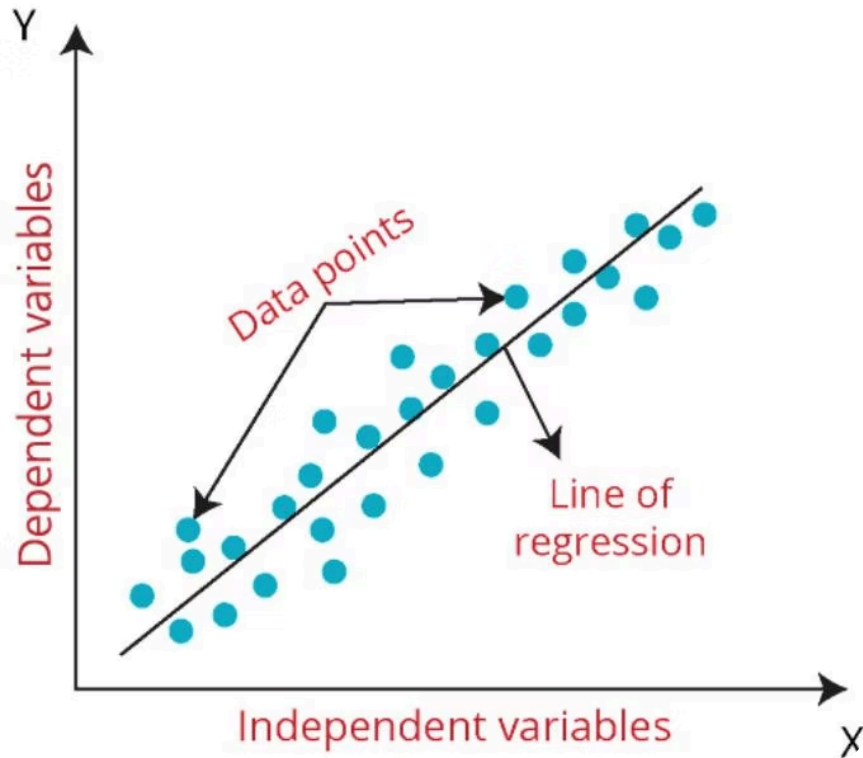
Model Evaluation & Validation

Cross-Validation, MSE, RMSE, MAE, R-Squared



Hyperparameter Tuning

GridSearchCV, RandomSearchCV, Sklearn Pipelines



What is Regression?

Regression is a **supervised learning** technique that models the relationship between a **dependent variable (Y)** and one or more **independent variables (X)** using a best-fit line or curve.

Dependent Variable (Y)

The output we want to predict — must be continuous.

Independent Variable (X)

The input features used to make predictions.

Regression Line

The best-fit line minimizing the error between actual and predicted values.

Applications of Regression



Oil & Gas Industry

Linear and non-linear regression models forecast global oil production using surface and subsurface data.



Marketing

Analyzes advertising effectiveness, predicts sales based on marketing spend, and segments customers.



Retail & Healthcare

Retail: demand forecasting, pricing optimization.
Healthcare: predicting patient outcomes and resource planning.



Real Estate

Predicts property prices based on location, size, amenities, and economic indicators.

Types of Regression

Linear Regression

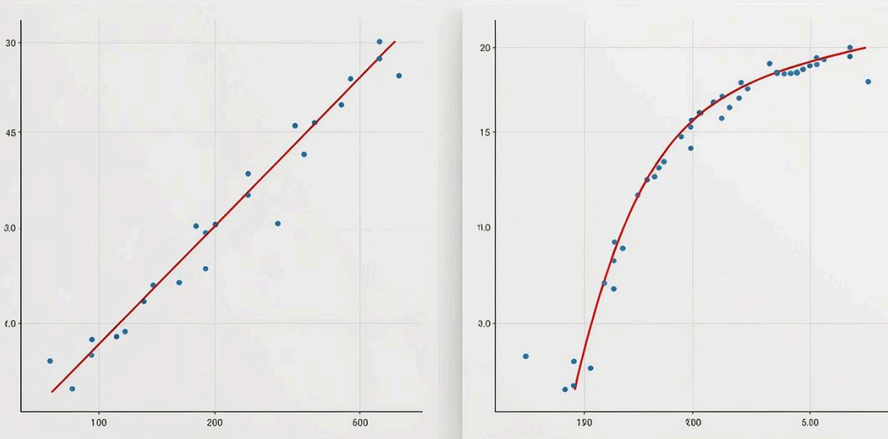
Finds a **straight-line** relationship between the dependent variable and one or more independent variables.

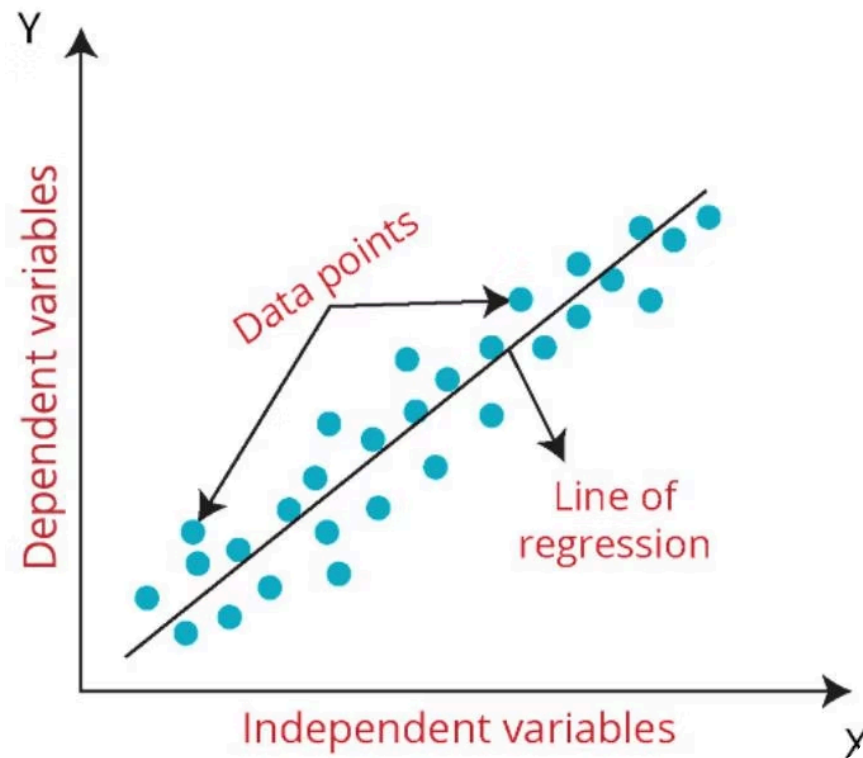
- Simple Linear Regression (1 predictor)
- Multiple Linear Regression (2+ predictors)
- Requires a linear relationship between variables
- Uses least squares to estimate coefficients

Non-Linear Regression

Finds a relationship using a **curve or complex shape** — used when data does not follow a straight line.

- Polynomial Regression (special case)
- Adds polynomial terms to capture complex patterns
- Better for non-linear datasets
- More flexible but risk of overfitting





Simple Linear Regression

Models the relationship between **one independent variable** and the dependent variable as a straight line.

The equation: $y = \beta_0 + \beta_1 x$ — where β_0 is the intercept and β_1 is the slope.

y

Dependent variable (what we predict, e.g., Sales)

x

Independent variable (input, e.g., TV Expenses)

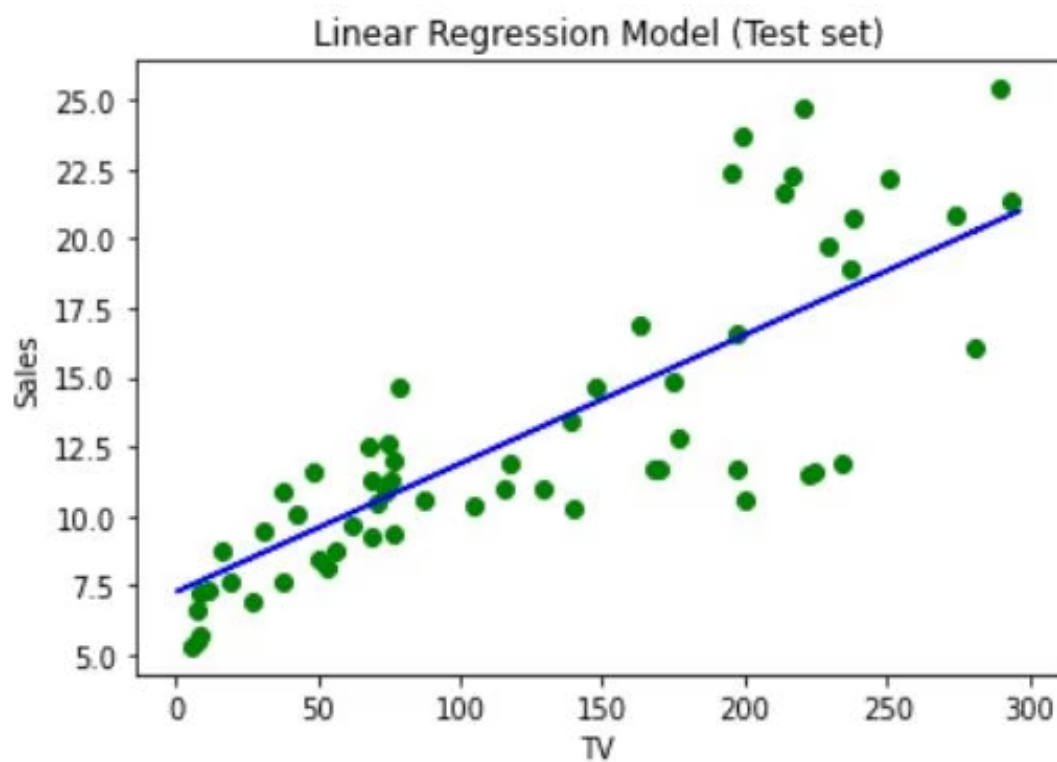
β_0 (Intercept)

Value of y when $x = 0$

β_1 (Slope)

Change in y for each unit increase in x

Simple Linear Regression: Python Implementation



```
# Step 1: Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

# Step 2: Load dataset
data = pd.read_csv("tvmarketing.csv")

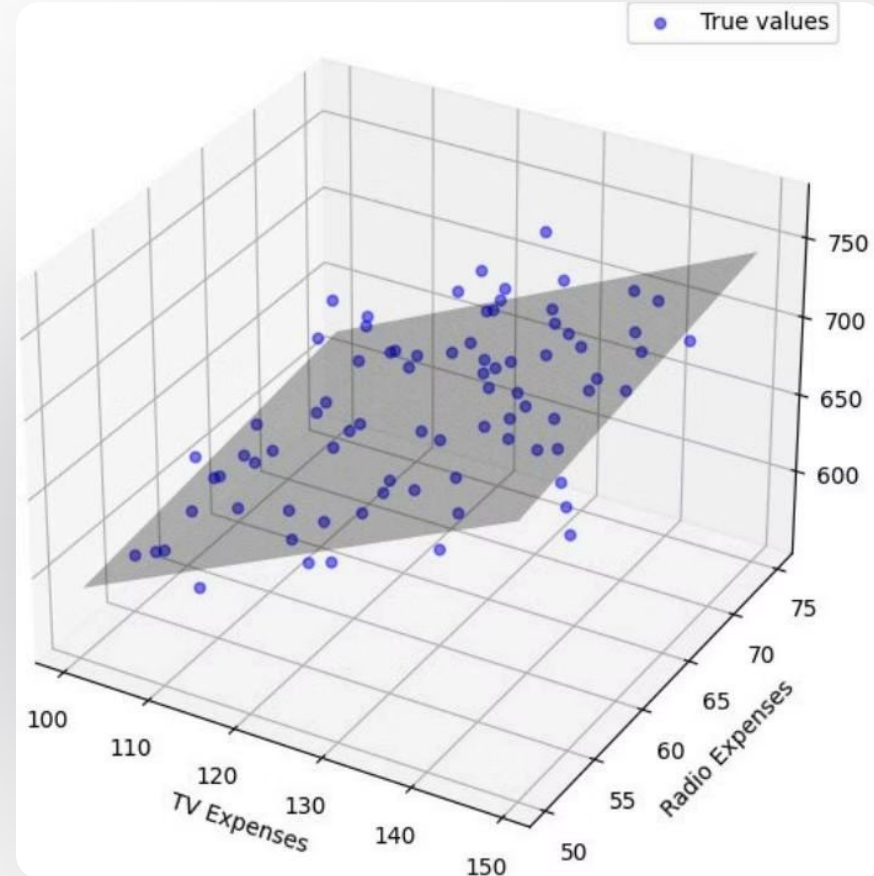
# Step 3: Define features (X) and target (y)
X = data.iloc[:, 0:1].values # TV Expenses (independent variable)
y = data.iloc[:, 1].values   # Sales (dependent variable)

# Step 4: Train-Test Split (70% train, 30% test)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

# Step 5: Fit the model
lin_reg = LinearRegression()
lin_reg.fit(X_train, y_train) # Model learns from training data

# Step 6: Visualize results
plt.scatter(X_test, y_test, color='green') # Test data points
plt.plot(X_train, lin_reg.predict(X_train), color='blue') # Regression line
plt.title('Linear Regression Model (Test set)')
plt.xlabel('TV')
plt.ylabel('Sales')
plt.show()
```

❗ The blue regression line shows a positive correlation — as TV spending increases, Sales increase. Green dots (test data) cluster around the line, confirming a good fit.



Multiple Linear Regression

Models the relationship between **two or more independent variables** and the dependent variable.

$$y = \beta_0 + \beta_1x_1 + \beta_2x_2 + \dots + \beta_nx_n$$
 — each β_j is the coefficient for predictor x_j .

The 3D plot above shows Sales predicted by both **TV Expenses** and **Radio Expenses**. The gray plane is the regression model — data points above/below the plane represent prediction errors. Multiple regression captures more variance than simple regression by using additional predictors.

Dr. Anilkumar V. Brahmane | Computer Engineering | Sanjivani College of Engineering

Overfitting, Underfitting & Bias-Variance Tradeoff

Overfitting

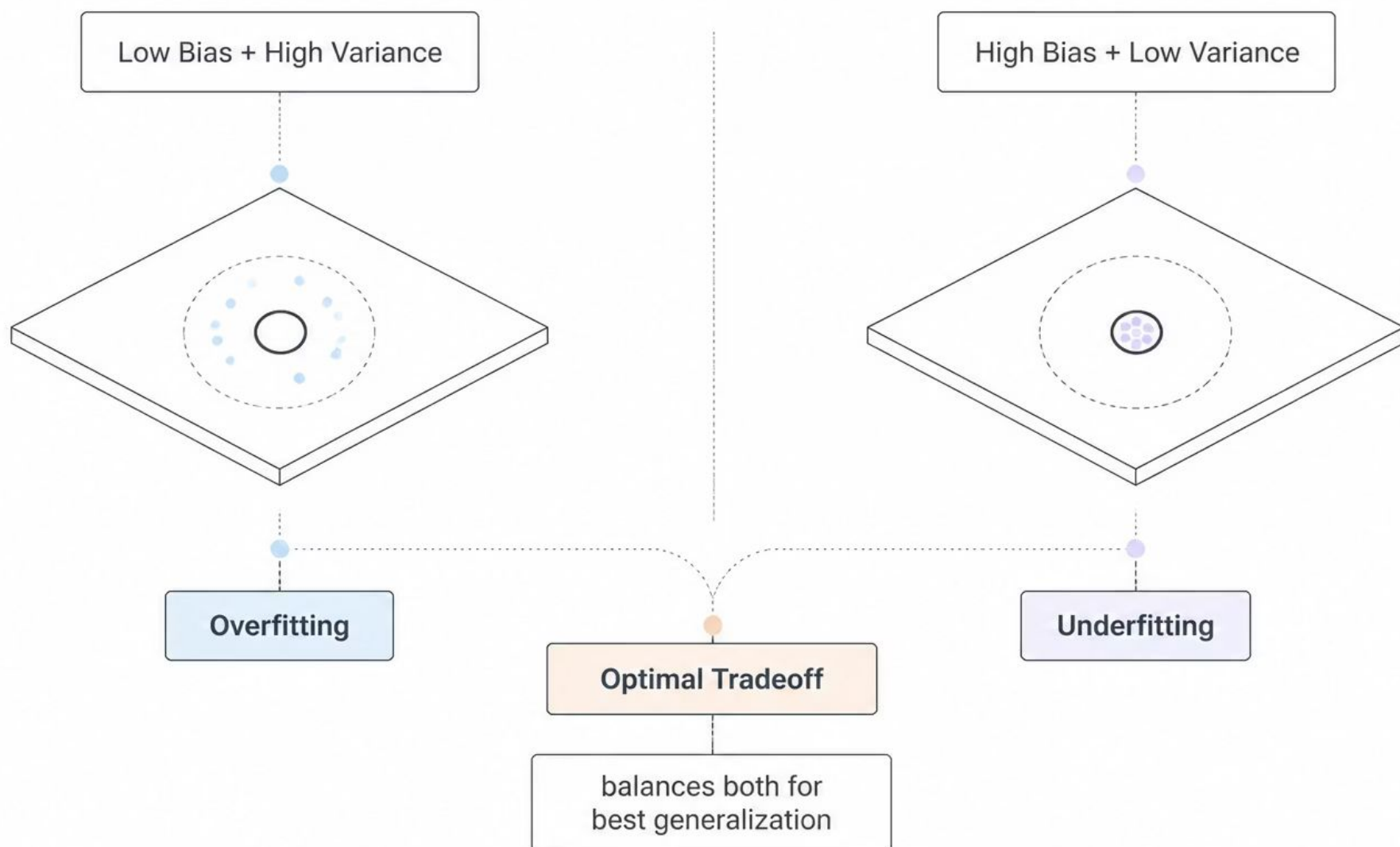
Model learns noise in training data. **High accuracy on training, poor on test.**
Cause: Too many parameters (too complex).

Underfitting

Model is too simple to capture patterns. **Poor accuracy on both training and test.**
Cause: Too few parameters.

Optimal Fit

Balanced model that generalizes well. **Good accuracy on both training and test data.** Goal of every ML model.



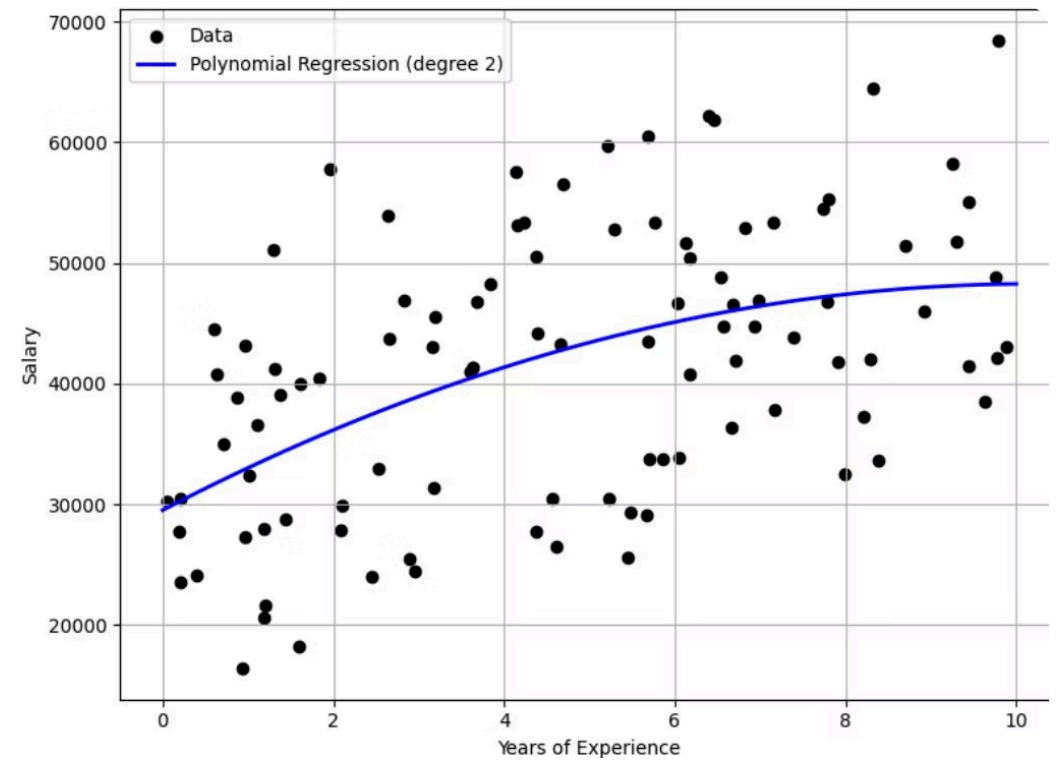
```
# Check for overfitting or underfitting
from sklearn.metrics import mean_squared_error, r2_score
y_train_pred = lin_reg.predict(X_train)
y_test_pred = lin_reg.predict(X_test)
print("Train R2:", r2_score(y_train, y_train_pred)) # Higher = better fit on training
print("Test R2:", r2_score(y_test, y_test_pred)) # Compare with train R2
```

Polynomial Regression (Non-Linear)

What is Polynomial Regression?

A subset of linear regression that adds **polynomial terms** (powers of x) to model non-linear relationships. The dataset used for training is non-linear.

- Captures complex, curved patterns in data
- Reduces error between actual and predicted values
- Degree controls the curve's flexibility

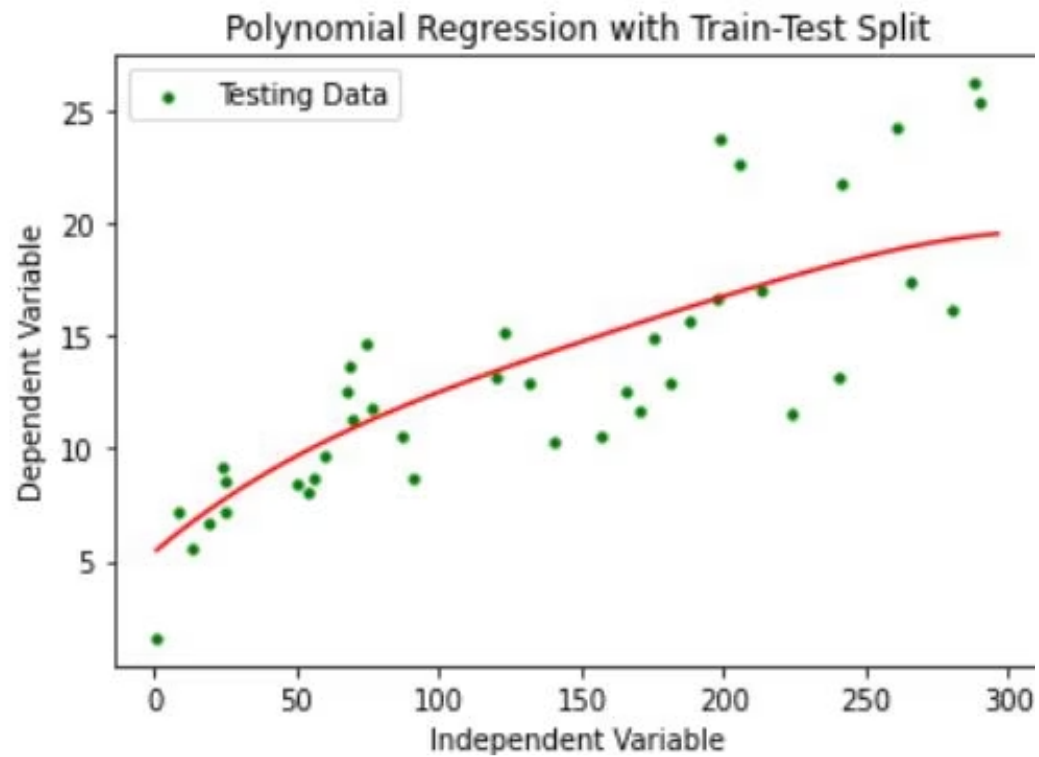


```
# Polynomial Regression: degree 4 on tvmarketing.csv
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

poly_features = PolynomialFeatures(degree=4) # Create polynomial features up to degree 4
X_train_poly = poly_features.fit_transform(X_train) # Transform training data
X_test_poly = poly_features.transform(X_test) # Apply SAME transform to test data

poly_model = LinearRegression()
poly_model.fit(X_train_poly, y_train) # Train on polynomial features
```

❗ The polynomial curve (degree 2) fits the Salary vs. Experience data much better than a straight line — capturing the slowing growth rate at higher experience levels.



Polynomial Regression: Results

The red polynomial curve fits the majority of the green test data points — confirming that **non-linear inputs require non-linear models**. A straight line would have missed the curved pattern entirely.

```
# Predict and visualize polynomial regression results
y_test_pred = poly_model.predict(X_test_poly)

# Create smooth curve for plotting
X_range = np.arange(X_train.min(), X_train.max(), 0.1).reshape(-1, 1)
plt.scatter(X_test, y_test, s=10, color='green', label='Testing Data')
plt.plot(X_range, poly_model.predict(poly_features.transform(X_range)), color='red')
plt.xlabel('Independent Variable')
plt.ylabel('Dependent Variable')
plt.title('Polynomial Regression with Train-Test Split')
plt.legend()
plt.show()
```

Performance Metrics for Regression

1

MSE — Mean Squared Error

Average of **squared differences** between actual and predicted values. Lower is better. Sensitive to large errors.

2

RMSE — Root Mean Squared Error

Square root of MSE — same units as the target variable, making it easier to interpret than MSE.

3

MAE — Mean Absolute Error

Average of **absolute differences**. Less sensitive to outliers compared to MSE and RMSE.

4

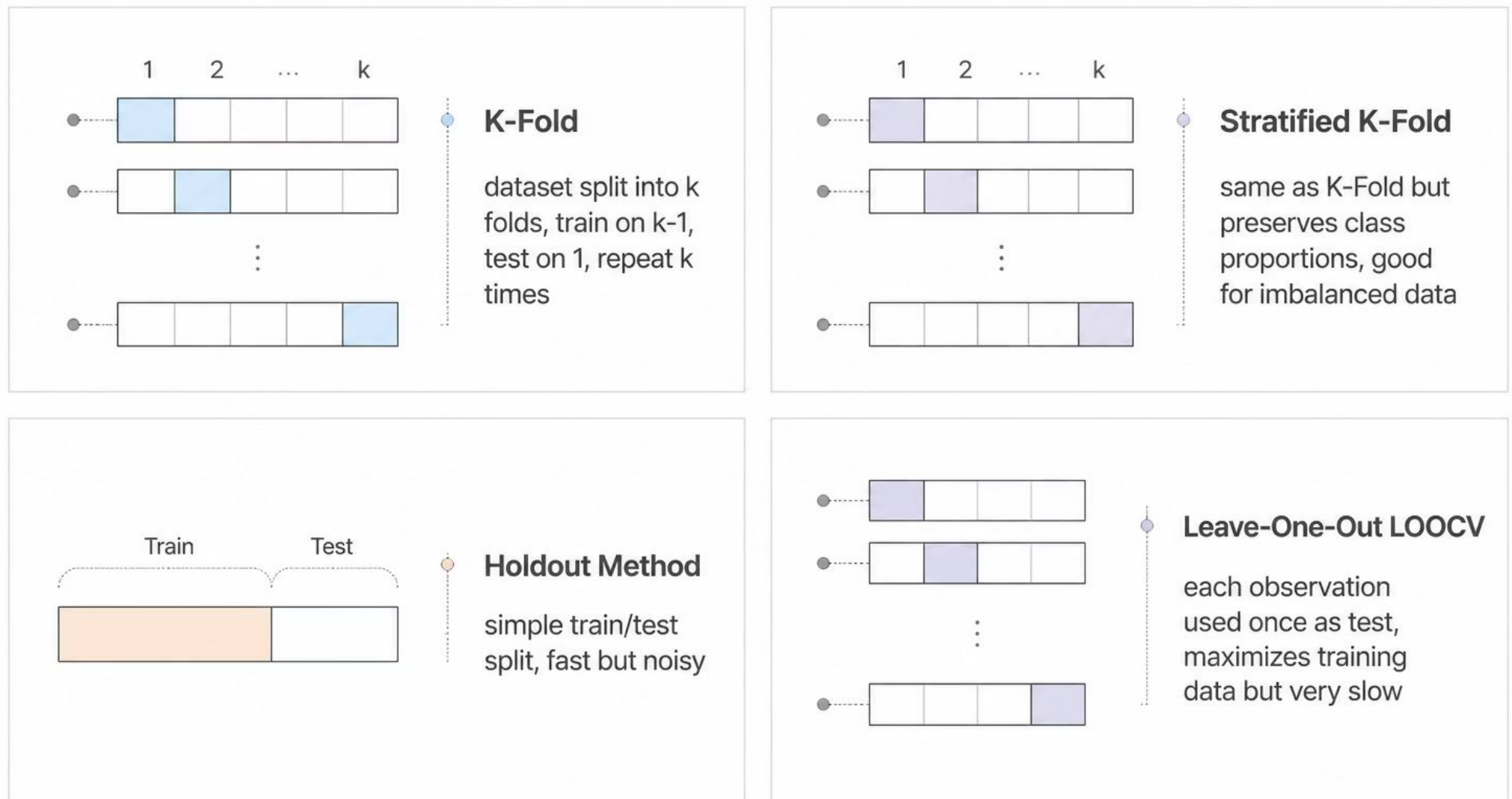
R-Squared (R^2)

Explains **how much variance** in Y is explained by X. Ranges from 0 (no fit) to 1 (perfect fit). Higher is better.

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
mse = mean_squared_error(y_test, y_pred) # Mean Squared Error
rmse = np.sqrt(mse) # Root Mean Squared Error
mae = mean_absolute_error(y_test, y_pred) # Mean Absolute Error
r2 = r2_score(y_test, y_pred) # R-Squared score
print(f"MSE: {mse:.2f}, RMSE: {rmse:.2f}, MAE: {mae:.2f}, R2: {r2:.2f}")
```

Cross-Validation Techniques

Cross-validation evaluates model performance on **unseen data** by dividing data into multiple folds — each fold serves as the validation set once, and results are averaged.



```
from sklearn.model_selection import KFold, cross_val_score, LeaveOneOut
```

```
# K-Fold Cross Validation (10 folds)
```

```
kf = KFold(n_splits=10, random_state=42, shuffle=True)
```

```
scores = cross_val_score(model, X_train, y_train,  
                          scoring='neg_mean_absolute_error',  
                          cv=kf, n_jobs=-1)
```

```
print("Mean MAE (K-Fold):", np.mean(np.abs(scores)))
```

```
# Leave-One-Out Cross Validation
```

```
loo = LeaveOneOut()
```

```
loocv_scores = cross_val_score(model, X_train, y_train,  
                               scoring='neg_mean_absolute_error',  
                               cv=loo, n_jobs=-1)
```

```
print("Mean MAE (LOOCV):", np.mean(np.abs(loocv_scores)))
```

❏ Note: Stratified K-Fold is typically used for classification. For regression, standard K-Fold is preferred.

Regularization in Regression

Regularization adds a **penalty term** to the model's objective function to prevent overfitting by discouraging large coefficients.

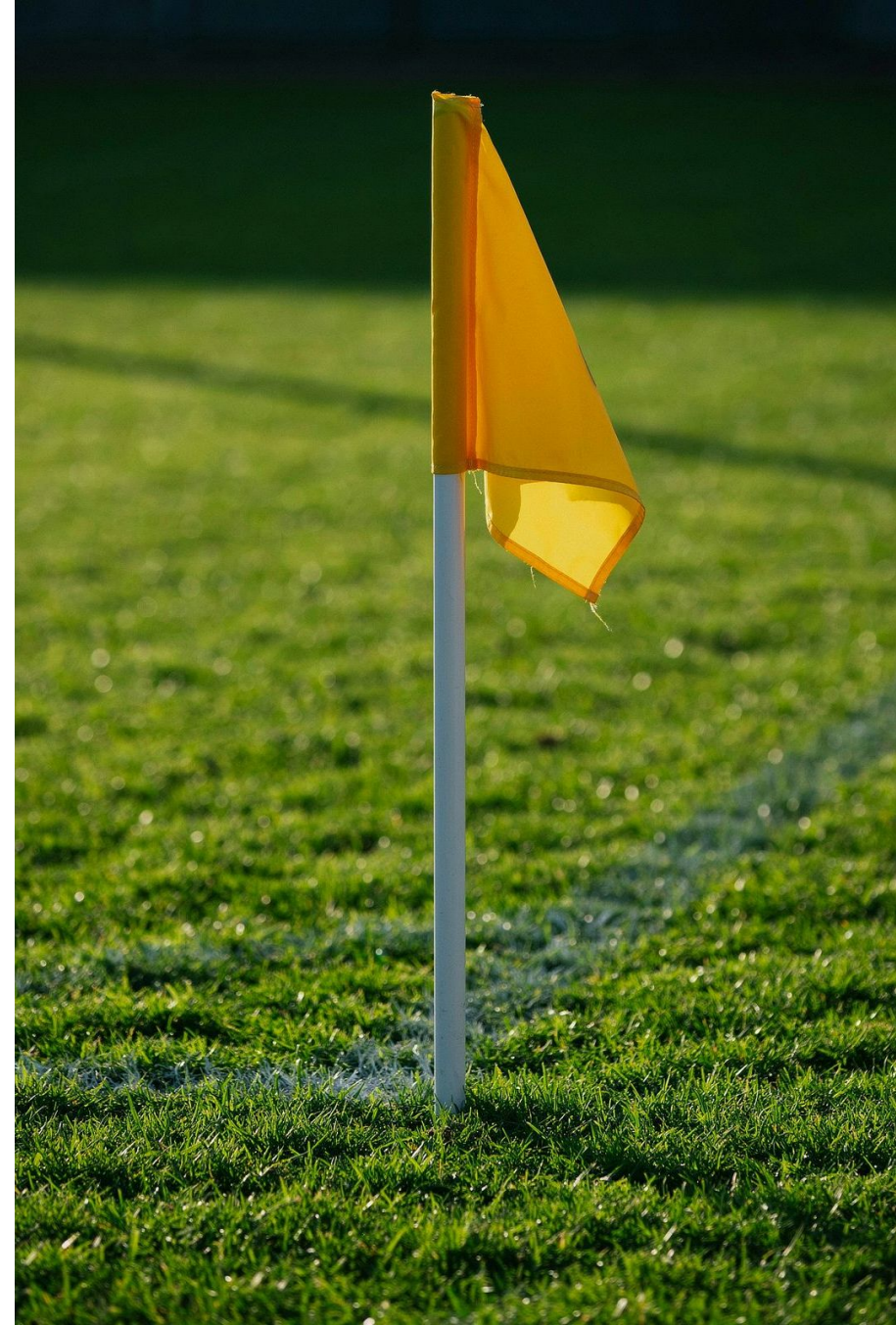
Alpha (Regularization Strength)

- **Higher alpha:** Stronger penalty → simpler model → may underfit but generalizes better
- **Lower alpha:** Weaker penalty → more complex model → risk of overfitting

Benefits of Regularization

- **Enhanced Generalizability:** Models perform well on unseen data
- **Reduced Complexity:** Promotes interpretability
- **Techniques:** Lasso (L1), Ridge (L2), ElasticNet (L1+L2)

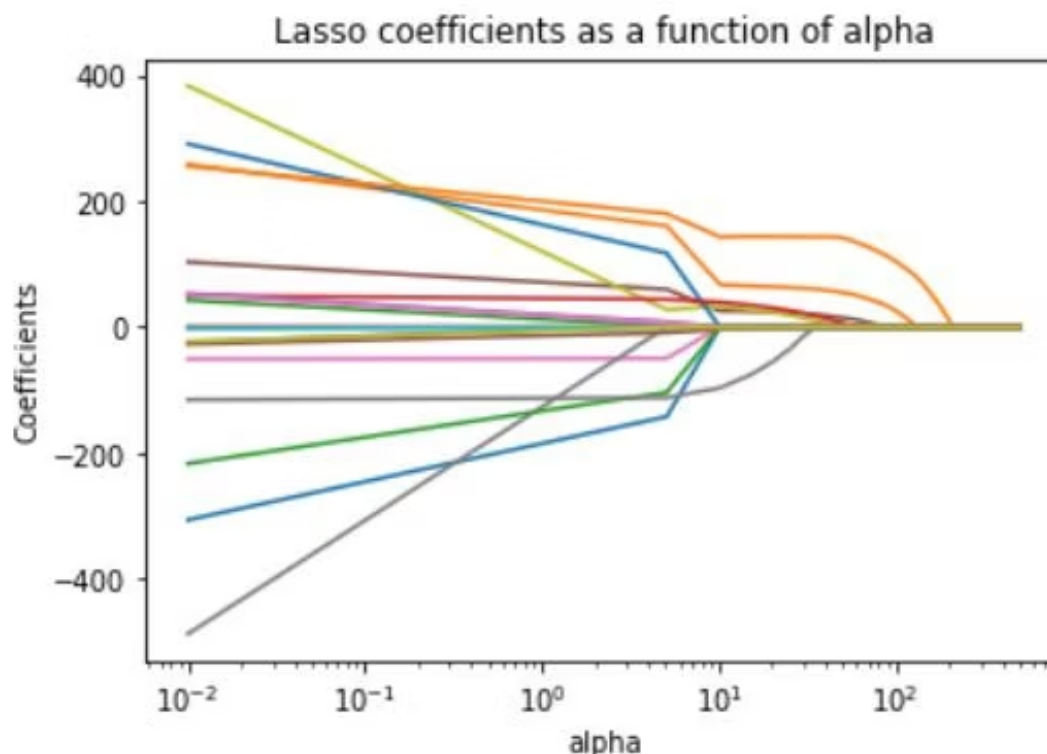
Dr. Anilkumar V. Brahmane | Computer Engineering | Sanjivani College of Engineering



Lasso Regression (L1 Regularization)

Lasso adds the **sum of absolute values of coefficients** as a penalty. It performs **feature selection** by shrinking some coefficients exactly to zero.

Objective: $\text{RSS} + \alpha \times \sum |\beta_j|$ — when $\alpha=0$: same as OLS; when $\alpha=\infty$: all coefficients are zero.



```
from sklearn.linear_model import Lasso, LassoCV
import numpy as np

# Fit Lasso with alpha=1
lasso_model = Lasso(alpha=1, max_iter=10000).fit(X_train, y_train)
print("Intercept:", lasso_model.intercept_) # Baseline prediction
print("Coefficients:", lasso_model.coef_) # Zero = feature removed!

# Use cross-validation to find best alpha automatically
lasso_cv = LassoCV(alphas=np.random.randint(0, 1000, 100),
                   cv=10, max_iter=10000, n_jobs=-1).fit(X_train, y_train)
print("Best alpha:", lasso_cv.alpha_) # Output: 3

# Retrain with best alpha
lasso_tuned = Lasso(alpha=lasso_cv.alpha_, max_iter=10000).fit(X_train, y_train)
```

❗ After tuning: Test MSE = 113,899 | Test R^2 = 39.56%. Features like Runs, RBI, CHmRun were shrunk to zero — Lasso eliminated them as unimportant.

Ridge Regression (L2 Regularization)

Ridge adds the **sum of squared coefficients** as a penalty. Unlike Lasso, it **shrinks coefficients but never forces them to zero** — no feature selection, but handles multicollinearity well.

Key Difference from Lasso

- Lasso: Some coefficients $\rightarrow$ exactly 0 (feature selection)
- Ridge: All coefficients $\rightarrow$ small but non-zero
- Ridge is better when all features are relevant
- Best for multicollinear (correlated) predictors

Results on Hitters Dataset

- Intercept: 552 (baseline salary)
- Train R^2 : 47.25% | Test R^2 : 38.06%
- Best alpha (CV): 203
- CRBI, CHits, CRuns had highest positive influence
- Division_W had the highest negative coefficient

```
from sklearn.linear_model import Ridge, RidgeCV

ridge_model = Ridge(alpha=1).fit(X_train, y_train)

# Cross-validation to find best alpha
ridge_cv = RidgeCV(alphas=np.random.randint(0, 1000, 100), cv=10).fit(X_train, y_train)
print("Best alpha:", ridge_cv.alpha_) # Output: 203

ridge_tuned = Ridge(alpha=ridge_cv.alpha_, max_iter=10000).fit(X_train, y_train)
pred_test = ridge_tuned.predict(X_test)
print("Test R²:", round(r2_score(y_test, pred_test)*100, 2))
```

ElasticNet Regression (L1 + L2)

ElasticNet **combines Lasso and Ridge** penalties — useful for highly collinear data where OLS fails. Two key parameters control the behavior.

alpha

Overall regularization strength. Higher = more shrinkage. Use log scale for tuning.

l1_ratio

Mix of L1 and L2. 0 = pure Ridge, 1 = pure Lasso. Values in between blend both.

Results

Best alpha: 0.0175 | Best l1_ratio: 0.06 | Test RMSE: 338.24 | Test R²: 39.3%

```
from sklearn.linear_model import ElasticNet, ElasticNetCV

# Initial ElasticNet model
elastic_net = ElasticNet(alpha=1.0, l1_ratio=0.5).fit(X_train, y_train)
print("Intercept:", elastic_net.intercept_) # Output: 518

# Tune using cross-validation
alphas = np.logspace(-2, 2, 100) # Range: 0.01 to 100
l1_ratios = np.linspace(0.01, 1, 100) # Range: 0.01 to 1 (avoid pure Ridge)

elastic_cv = ElasticNetCV(alphas=alphas, l1_ratio=l1_ratios,
                          cv=10, max_iter=10000).fit(X_train, y_train)
print("Best alpha:", elastic_cv.alpha_) # Output: 0.0175
print("Best l1_ratio:", elastic_cv.l1_ratio_) # Output: 0.06

elastic_tuned = ElasticNet(alpha=elastic_cv.alpha_,
                           l1_ratio=elastic_cv.l1_ratio_,
                           max_iter=10000).fit(X_train, y_train)
```

Hyperparameter Tuning: GridSearchCV & RandomSearchCV

Grid Search

Exhaustively tests **all combinations** of specified hyperparameters. Guaranteed to find the best within the grid, but slow for large grids.

Random Search

Randomly samples combinations from specified distributions. **Faster** than grid search, often finds near-optimal results with fewer evaluations.

```
from sklearn.model_selection import GridSearchCV, RepeatedKFold
from sklearn.linear_model import Ridge

# Define cross-validation strategy
cv = RepeatedKFold(n_splits=10, n_repeats=3, random_state=1)

# Define hyperparameter grid to search
grid = {'alpha': np.arange(0, 1, 0.1)} # Test alpha values: 0.0, 0.1, 0.2, ..., 0.9

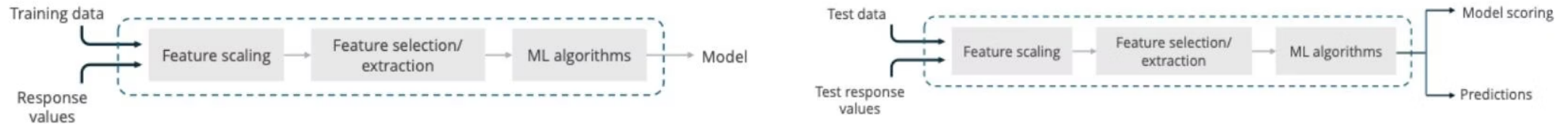
# Create model and GridSearchCV
model = Ridge()
search = GridSearchCV(model, grid,
                      scoring='neg_mean_absolute_error',
                      cv=cv, n_jobs=-1) # n_jobs=-1 uses all CPU cores

results = search.fit(X_train, y_train)
print("Best MAE:", results.best_score_) # Output: -238.285
print("Best Config:", results.best_params_) # Output: {'alpha': 0.9}

# Use best alpha to retrain and predict
best_model = Ridge(alpha=results.best_params_.get('alpha')).fit(X_train, y_train)
y_pred = best_model.predict(X_test)
print("Test R²:", r2_score(y_test, y_pred)) # Output: 0.38
```

Sklearn Pipelines

Pipelines combine **preprocessing and modeling steps** into a single, streamlined process — cleaner code, fewer bugs, and easier to deploy.



```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer

# Pipeline for numeric columns: fill missing values → standardize
num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')), # Fill NaN with median
    ('std_scaler', StandardScaler())             # Scale to mean=0, std=1
])

# ColumnTransformer: apply different steps to numeric vs categorical columns
preprocessor = ColumnTransformer([
    ('num', num_pipeline, housing_num),          # Numeric columns
    ('cat', OneHotEncoder(), housing_cat)        # Categorical columns
])

# Full pipeline: preprocessing + model in one object
full_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('model', LinearRegression())
])
full_pipeline.fit(X_train, y_train) # Fit everything in one step
predictions = full_pipeline.predict(X_test)
```



Key Takeaways

1 Regression Types

Use **Simple/Multiple Linear Regression** for linear data; **Polynomial Regression** for non-linear patterns.

2 Model Evaluation

Always evaluate using **MSE, RMSE, MAE, and R^2** . Use **K-Fold Cross-Validation** for robust performance estimates.

3 Regularization

Lasso for feature selection (zeros out coefficients); **Ridge** for multicollinearity; **ElasticNet** combines both.

4 Hyperparameter Tuning & Pipelines

Use **GridSearchCV** or **RandomSearchCV** to find optimal alpha. Use **Sklearn Pipelines** for clean, production-ready code.

Dr. Anilkumar V. Brahmane | Computer Engineering | Sanjivani College of Engineering